

HELIX PRIME · ENGINEERING

Helix Codex App — System Analysis and Design

Architecture, governance and design of the local-first operations shell

Abstract: Helix Codex App is the daily-use product shell over the governed Helix Codex OS core. It runs entirely on hardware the operator controls: a single FastAPI process bound to loopback, a SQLite store, and no third-party origin at runtime. This document analyses the system as built — its context and boundaries, its twelve feature modules, the guard chain every request passes through, the two-key rule that governs any change to shared state, the data model, the bridges into the governed core, and the release gate that decides what may ship.

Helix Prime Engineering

September 2026

Executive summary

Helix Codex App is the daily-use product shell that sits over the governed Helix Codex OS core. It is not a dashboard bolted onto an API: it is the surface through which an academy's staff do their work — tasks, documents, conversation, attendance, approvals — while every write that touches shared state passes through a governance layer that predates the interface.

The system was built against one constraint that shapes almost every other decision: **it must run on hardware the operator controls, and it must be able to prove what it did.** That constraint rules out a hosted multi-tenant service, rules out runtime calls to third-party origins, and rules out any feature whose audit trail would have to be taken on trust.

The design thesis: a single process, bound to loopback, owning one store — where every change to shared state is proposed by one person and approved by another, and the record of both decisions is written to a hash-chained ledger the operator can re-verify at will.

What this document covers

Sections 2 to 4 establish the boundary and the shape: who the actors are, what the app owns, how the code is layered, and what happens to a request from arrival to response. Sections 5 and 6 cover the two mechanisms that carry the governance claim — the guard chain that decides whether a caller may act at all, and the two-key rule that decides whether a change to shared state may land. Sections 7 to 11 cover data, integration, runtime, navigation and release. Sections 12 and 13 record the honest limits of the current build and what a next phase should address.

The system in one table

Question	Answer
What is it?	Server-rendered web application, installable as a PWA
Who runs it?	One operator, on one machine, on their own network
What does it store?	One SQLite file (28 tables) plus per-account memory store files
How is it reached?	Loopback only — 127.0.0.1 by default, enforced at startup

What governs writes?	Guard chain, permission matrix, two-key approval, audit chain
What is simulated?	Everything: data_mode is simulated_realistic, no live connectors
What may ship?	Whatever the release gate classifies; currently app_pilot

System context and boundaries

The app has five classes of human actor and one machine boundary. Everything else is delegated. Distinguishing what the app owns from what it merely fronts is the first useful cut in the analysis, because the governance guarantees live on the delegated side and the usability guarantees live on the owned side.

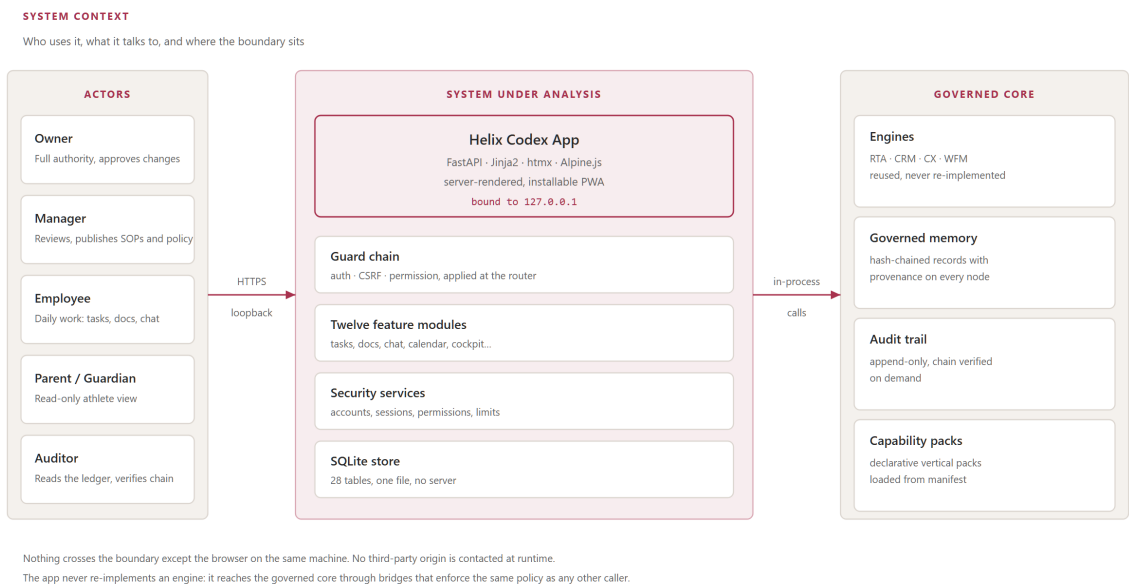


Figure 1: The system context. Actors reach the app over loopback; the app reaches the governed core in-process through bridges. No third-party origin appears anywhere in the runtime path.

Actors and their authority

Actor	Reaches	Authority	Cannot
Owner	Everything	Full permission set; sole approver of packs	—

Manager	All work, governance and admin	Publish SOPs and policy; review proposals	Manage packs; edit accounts outside scope
Employee	Work, docs, own memory	Propose changes to their own practice	Review proposals, view ops or cockpit
Contractor	Work and docs	Same daily surface as an employee	Propose, review, or see governance
External	Chat and scoped documents	Participate in conversation	Write documents, punch, see tasks

What the app owns, and what it delegates

The app **owns** the store schema, the session and account model, the permission matrix, the screen layer, and the shape of every record it writes. It **delegates** all computation to the governed core: adherence, churn risk, coverage and pipeline figures are produced by engines that the app calls rather than re-implements.

This division matters because it makes drift structurally unlikely. A cockpit tile cannot quietly disagree with the engine that owns the number, because the tile is rendering the engine's return value, not a second implementation of it. Where the app does compute — counts, filters, formatting — the result is presentation, not judgement.

Boundary rule: the app may not open an outbound connection at runtime. Fonts are self-hosted, scripts are vendored, and the only network interface is the loopback socket the browser talks to.

Architecture

Layering

Six layers, each depending only on the one beneath it. The discipline is enforced by convention rather than by a framework, so the value is in the naming: a reader can tell from a file's path which layer it belongs to and therefore what it is allowed to do.

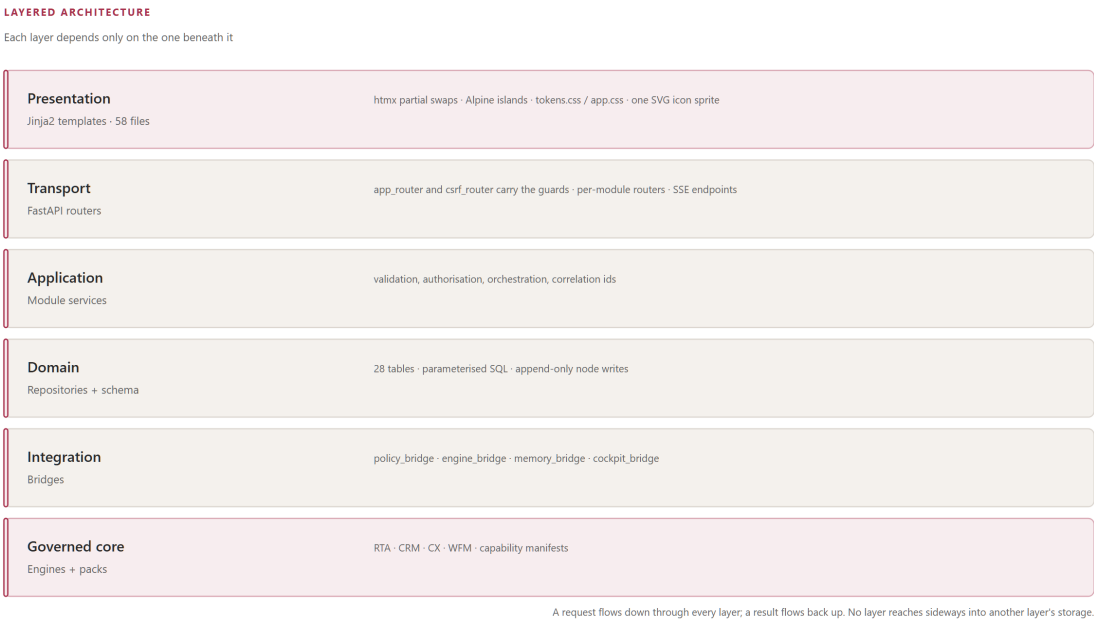


Figure 2: Layering. A request descends through every layer; a result ascends. No layer reaches sideways into another layer's storage.

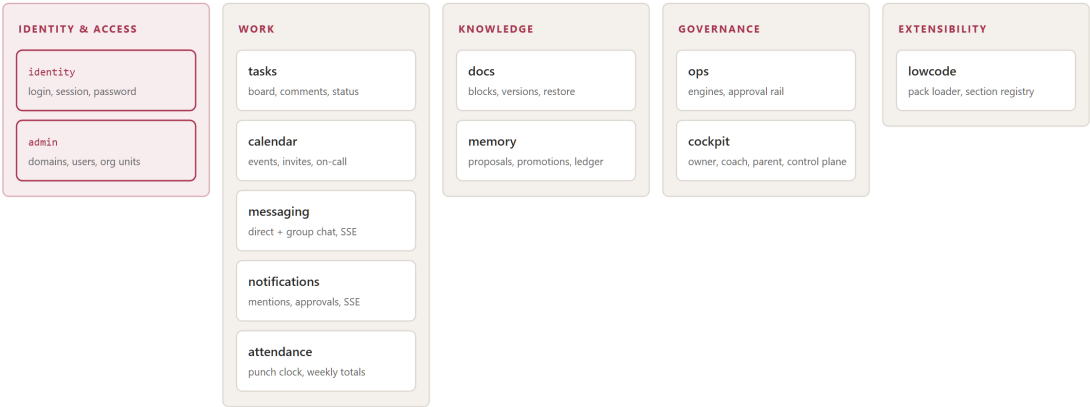
The transport layer is where the guards are attached. Two routers carry them: **app_router** applies the account guard to everything mounted beneath `/app`, and **csrf_router** adds the CSRF check for mutating routes. A feature module never installs its own authentication, which is what makes the guarantee uniform.

Component decomposition

Twelve feature modules, grouped by the question they answer. Each has the same internal shape — a router that guards and delegates, a service that owns the rules, and a repository that owns the SQL. Where a module needs a repository it does not own, it goes through a service or a bridge rather than importing the repository directly.

COMPONENT DECOMPOSITION

Twelve feature modules, each shaped router → service → repository



Every module follows the same shape: a router that only guards and delegates, a service that owns the rules, a repository that owns the SQL.
A module may not import another module's repository — cross-module reads go through services or through a bridge.
Band height follows the number of modules, so the map shows real weight rather than a padded grid.

Figure 3: Component decomposition. Band height reflects the number of modules in each group, so the map shows real weight rather than a padded grid.

Module	Responsibility	Principal tables
identity	Login, session issue and revocation, password change	sessions, login_events
admin	Domains, org units, accounts, limits, evidence export	domains, org_units, accounts, account_limits
tasks	Board, assignment, status transitions, comments	tasks, task_comments
calendar	Events, invitations, on-call rota	events, event_attendees, oncall_shifts
messaging	Direct and group conversation, streaming delivery	conversations, conversation_members, messages
notifications	Mentions, approvals, delivery state	notifications
attendance	Punch clock, weekly totals, records	punch_records
docs	Block documents, versions, restore, knowledge base	documents, doc_blocks, document_versions, files
memory	Proposals, evaluation, promotion, ledger verification	nodes, proposals, proposal_reviews, promotions, memory_stores
ops	Engine status, workflow submission, approval rail	— (delegated to the core)

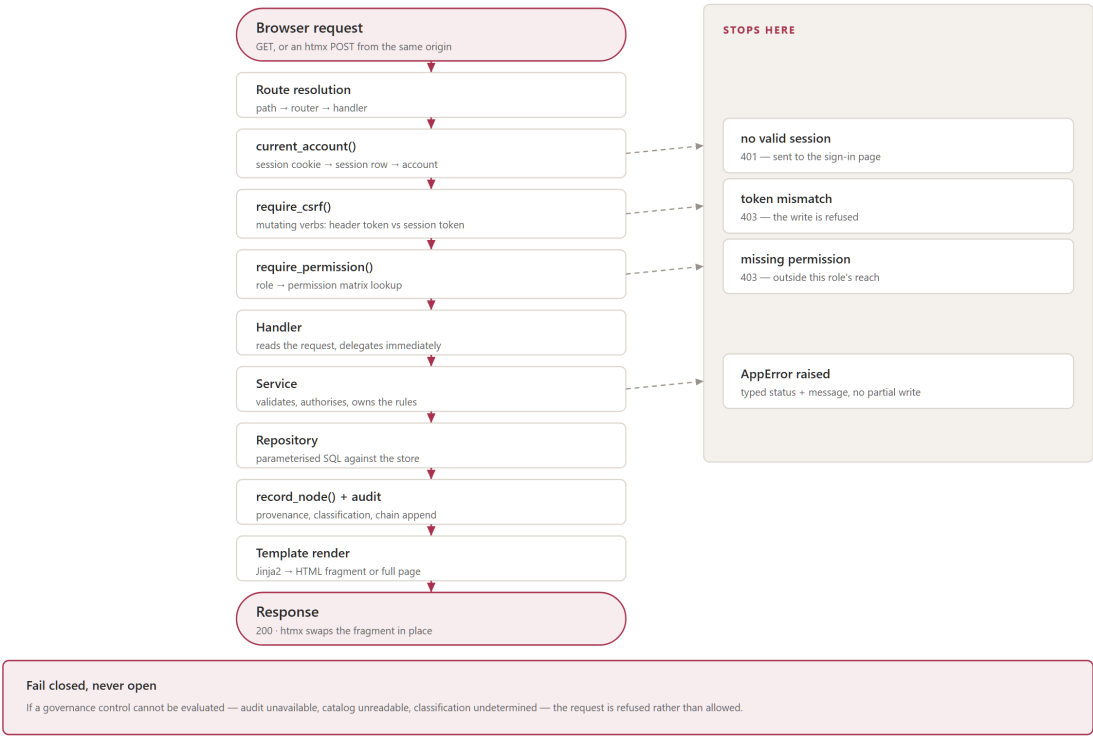
cockpit	Owner, coach, parent and control-plane views	— (computed by packs)
lowcode	Capability pack loading and section registry	capability_packs, sections

Request lifecycle

Every request to `/app` passes through the same sequence before a handler sees it. The ordering is deliberate: identity is established before the CSRF token is compared, and the token is compared before the role is consulted. A request that fails at any step never reaches application code, so no individual module has to defend itself against an unauthenticated caller.

REQUEST LIFECYCLE

The guard chain every /app request passes through, and exactly where it can stop



The order is deliberate: identity before token, token before role. A request that fails any step never reaches a handler, so no module has to defend itself.

Every mutating route `require_csrf` at its own decorator, so a new route cannot inherit protection by accident.

Provenance is written in the same transaction as the change: tenant, client, correlation id, data mode, basis and classification travel with the record.

Data mode is `simulated_realistic` throughout — this build opens no live connector write path.

Figure 4: The guard chain, and the four points at which a request can stop. Failure branches are drawn to the right of the step that produces them.

Why the guards sit at the router, not in the handler

Applying a guard per handler invites the one failure mode that matters: a new route added without it. Attaching the account guard to a parent router means the default is protected and the exception is explicit — the health probe is the only route that opts out, and it is declared in one place.

The CSRF check is attached per mutating route rather than to the parent, because it must not fire on reads. That is a deliberate asymmetry: it costs a repeated decorator, and it buys the property that a write cannot inherit protection by accident — it has to state it.

Fail closed, never open

If a governance control cannot be evaluated — the audit trail is unreachable, the role catalog will not parse, a record's classification cannot be determined — the request is refused. The system does not degrade into an ungoverned mode, because an unlogged write is worse than a refused one.

This is the single most important behavioural choice in the build, and it was made explicitly during hardening. Earlier revisions allowed several controls to skip silently when their dependency was missing; those paths now raise. The observable consequence is that a misconfigured deployment refuses work rather than appearing to succeed.

Security architecture

Authentication and session

Authentication is deliberately unremarkable, which is the point. A browser presents credentials once; the system verifies a hash, issues a session, and thereafter identifies the caller by a cookie. The raw session token is never stored — only its hash — so a copy of the store does not yield usable sessions.

AUTHENTICATION AND SESSION

How a browser becomes an identified caller, and what happens when it should not

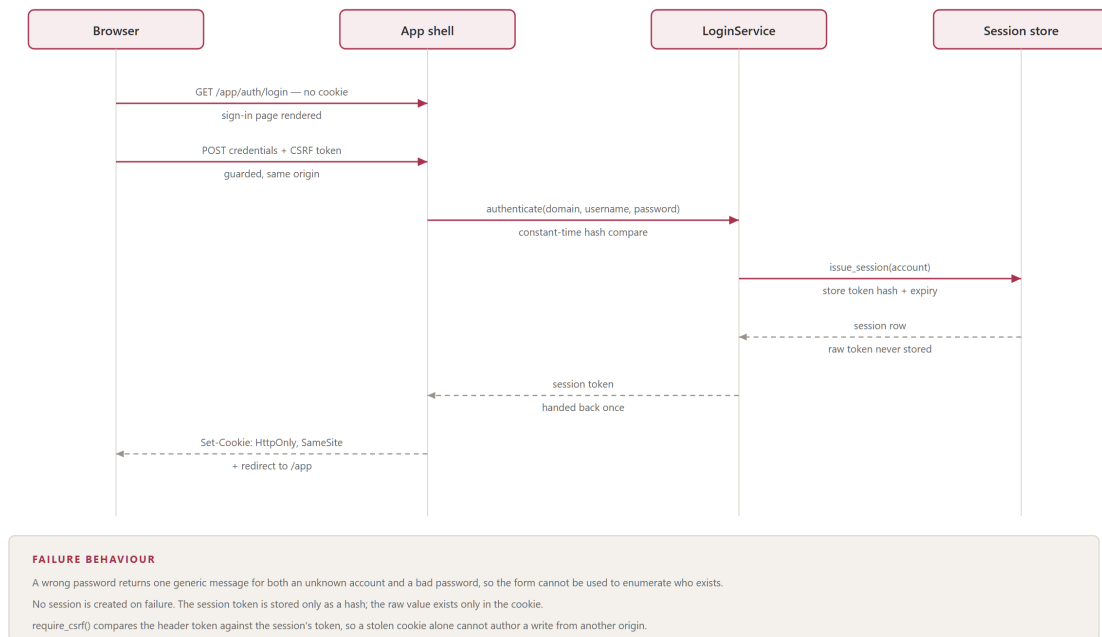


Figure 5: Authentication and session issue. Returns are drawn dashed; the raw token is handed back once and never persisted in clear.

Two properties are worth naming because they are easy to lose. First, a failed login returns the same message for an unknown account and a wrong password, so the form cannot be used to enumerate who exists. Second, the CSRF token is compared against the session's own token, so possessing a cookie is not by itself sufficient to author a write from another origin.

Authorisation

Authorisation is a data problem, not a control-flow problem. Thirteen permissions are checked against five roles through a matrix, and routers declare which permission they require. Changing what a role may do is an edit to that matrix rather than a hunt through conditionals.

AUTHORISATION MODEL

Thirteen permissions checked against five roles — the matrix is data, not branching code

	owner	manager	employee	contractor	external
chat.use	☒	☒	☒	☒	☒
docs.read	☒	☒	☒	☒	☐
docs.write	☒	☒	☒	☒	
tasks.use	☒	☒	☒	☒	
calendar.use	☒	☒	☒	☒	
attendance.punch	☒	☒	☒	☒	
notifications.use	☒	☒	☒	☒	☒
memory.propose	☒	☒	☒		
memory.review	☒	☒			
ops.view	☒	☒			
cockpit.view	☒	☒			
admin.users	☒	☐			
packs.manage	☒				

☒ allowed ☐ allowed, scoped to the caller's own records an empty cell is denied — the default is refusal

ENFORCEMENT

Routers declare `require_permission("docs.write")` once; the matrix above decides the answer. Changing a role's reach is a data edit, not a code change.

An unknown role resolves to the empty row, so a typo fails closed rather than granting the union of everything.

Figure 6: The permission matrix as built. A filled cell is allowed, a dashed cell is allowed but scoped to the caller's own records, and an empty cell is denied.

Two details carry more weight than they appear to. An unknown role resolves to the empty row, so a typo in a role identifier produces a caller who can do nothing rather than one who can do everything. And the matrix is not the only gate: routes that touch the governance surface additionally check a capability derived from the role catalog, so authority has two independent sources that must agree.

The session boundary

Sessions expire, are revoked on sign-out, and are re-checked on every request rather than trusted from a cached identity. A revoked session fails immediately even if its cookie is still presented, which is what makes sign-out meaningful on a shared machine.

Governed memory and the two-key rule

The product's central claim is that anything changing shared state is proposed by one person and approved by another, with both decisions recorded. That claim is implemented as a state machine with an explicit exclusion, not as a policy paragraph.

Proposal lifecycle

PROPOSAL LIFECYCLE

Any change to shared state is proposed, evaluated, and approved by someone else



RULES THE STATE MACHINE ENCODES

Only the author may evaluate their own proposal — you measure your own idea against the evidence before anyone else spends time on it.
Only a different person may approve it. The review queue excludes your own proposals and any author sharing your account, so the two keys cannot be the same hand.
Rollback is offered only from approved or already-rolled-back states; a rejected proposal is closed, not silently revived.
Every transition appends a record, so the ledger shows who moved it, when, and on what evidence.

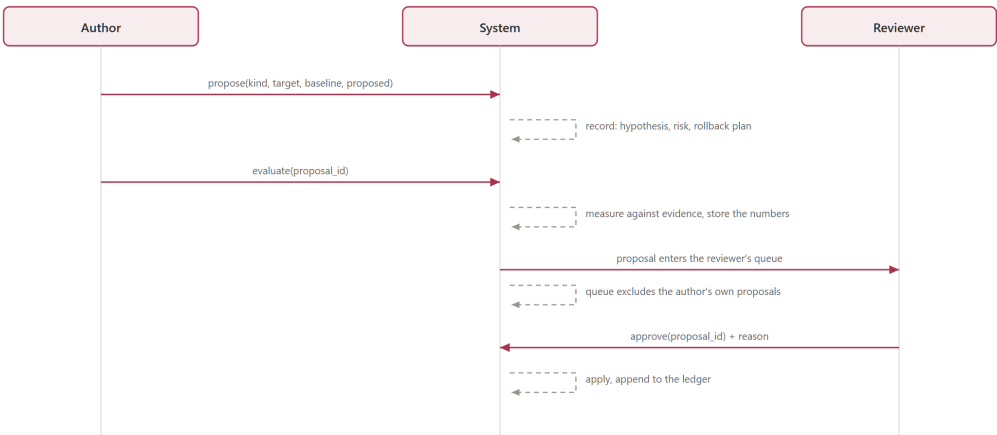
Figure 7: The proposal state machine. Evaluation is the author's job; approval is somebody else's; rollback is available only from a state that could plausibly need it.

Separating *evaluate* from *approve* is the design decision that makes the rest work. An author must measure their own proposal against the evidence before anyone else spends time on it — which removes the review queue as a place to dump untested ideas — and only then does it become eligible for a second person's decision.

Why the two keys cannot be the same hand

THE TWO-KEY RULE

One person proposes; a different person approves. Neither key alone moves shared state



WHY THIS IS STRUCTURAL, NOT PROCEDURAL

The exclusion lives in the query that builds the queue, not in a policy document: an author cannot see their own proposal in the list they approve from.
Authority comes from the role catalog, never from a hardcoded name. If the catalog cannot be read, the check raises rather than allowing.
A rejection carries a reason. A rollback carries a reason. Both are recorded, so the ledger explains not just what happened but why.

Figure 8: The two-key sequence. The exclusion lives in the query that builds the review queue, so an author cannot see their own proposal in the list they approve from.

The exclusion is enforced where the queue is built rather than where the approval is processed. That is stronger: an author never sees the affordance at all, so the control does not depend on the button being correctly disabled. Authority for the decision is drawn from the role catalog rather than from a hardcoded name, and if the catalog cannot be read the check raises rather than passing.

Provenance on every record

Every governed record carries the same envelope. This is not metadata for its own sake: it is what allows a reader to answer, without trusting the application, where a number came from and under what conditions it was produced.

Field	Purpose
tenant_id / client_id	Which operator and which client the record belongs to
correlation_id / causation_id	Which request produced it, and which earlier record it follows from
classification	How sensitive the body is, decided before storage
data_mode	Whether the content is simulated or live — simulated_realistic throughout this build
basis / sources	What the record rests on, and the references that support it
evidence_refs	Pointers to the evidence a reviewer can inspect

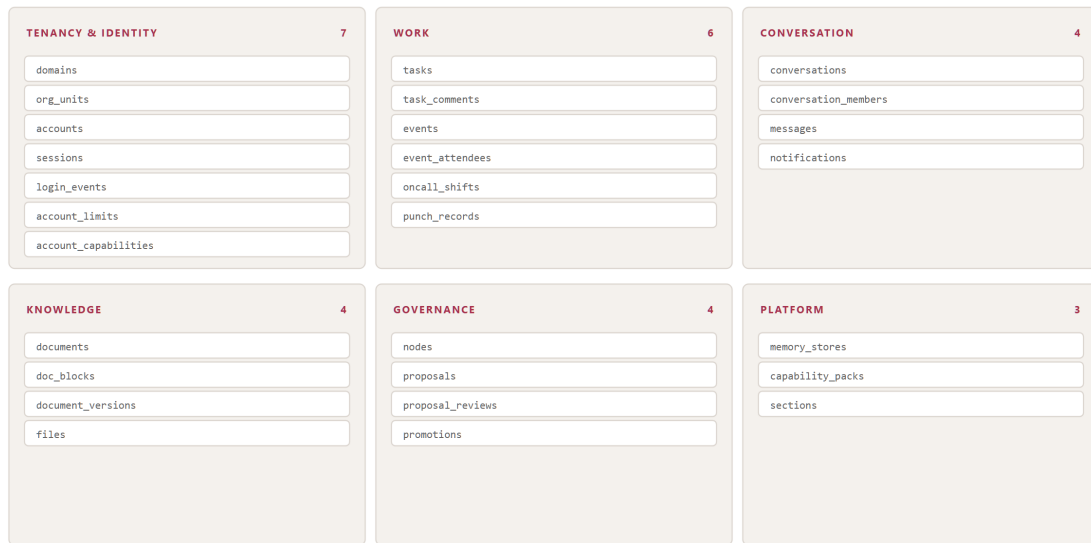
Records are written as nodes on an append-only chain. Each store file is hash-chained per account, and *verify_store_file()* re-walks the chain on demand — so tampering is detectable by the operator rather than asserted by the vendor. The Memory screen exposes that verification as a user action.

Data architecture

One SQLite file holds twenty-eight tables. The grouping below is by the question each table answers rather than by the module that happens to write it, because several tables are read across module boundaries and the module layout would misrepresent those relationships.

DATA ARCHITECTURE

Twenty-eight tables in one SQLite file, grouped by the question each answers



One append-only spine: every governed write lands in nodes with tenant, client, correlation id, classification and provenance.
 memory_stores records where each account's hash-chained store file lives; verify_store_file() re-walks the chain on demand.
 The schema is owned by Alembic; check_app_migration_drift.py fails the build if the store and the migration head disagree.
 SQLite in WAL mode is sufficient because a single process owns the file — the deployment target is one operator, not a fleet.

Figure 9: The store grouped by domain. Counts show the weight of each area; the governance group is small in table count but is the spine every other group writes through.

Design decisions worth stating

SQLite is sufficient, and that is a consequence of the deployment shape, not an accident. One process owns the file, so there is no concurrent-writer problem to solve and no need for a server. The trade is a ceiling on scale, and that trade is acceptable for the stated target of one academy.

The schema is owned by Alembic, and drift is a build failure. A check compares the live store against the migration head and fails if they disagree, so a schema change cannot reach a release without its migration.

Memory stores live outside the database on purpose. Each account's governed memory is a hash-chained file under a resolved path, recorded in the *memory_stores* table. Keeping the chain in a file means it can be verified, copied and diffed with ordinary tools rather than requiring the application to be running.

Integration architecture

The app reaches the governed core through four bridges. Every bridge does the same three things: it converts the app's account into the core's identity shape, it asks the policy gate whether this caller may make this call, and it invokes the core function. Only then does it return a value.

INTEGRATION ARCHITECTURE

The app reaches the governed core only through bridges that re-authorise every call

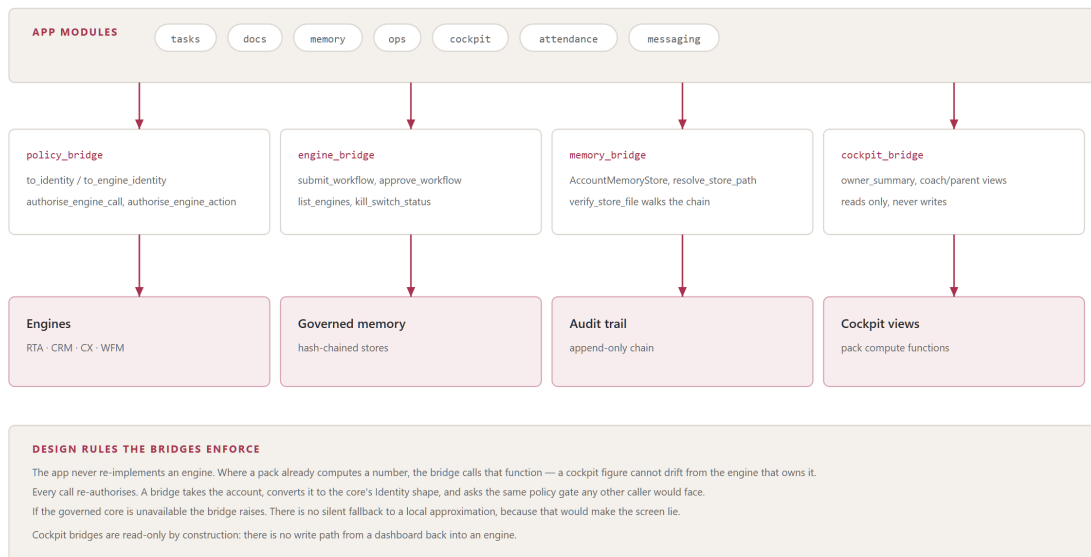


Figure 10: The four bridges and what each is for. Every path from an app module to the core passes through a policy check, so the app cannot become a privileged back door.

Rules the bridges exist to enforce

Re-authorisation on every call. The bridge does not assume that because the caller reached a screen, they are entitled to what is behind it. The same policy gate that any other caller would face is applied again.

No local approximation. If the core is unavailable the bridge raises. The tempting alternative — compute a plausible number locally and show that — is explicitly rejected, because a screen that shows an approximation without saying so is worse than a screen that admits it cannot answer.

Read-only where the surface is read-only. The cockpit bridges have no write path back into an engine. A dashboard that can change state is a dashboard that needs its own approval rail, and the decision here was to not build one.

Runtime and deployment topology

The whole system is one process, one store and one machine. There is no worker pool, no message broker and no second writer, which removes an entire category of consistency question before it can be asked.

RUNTIME TOPOLOGY

One process, one store, one machine — the boundary is the loopback interface

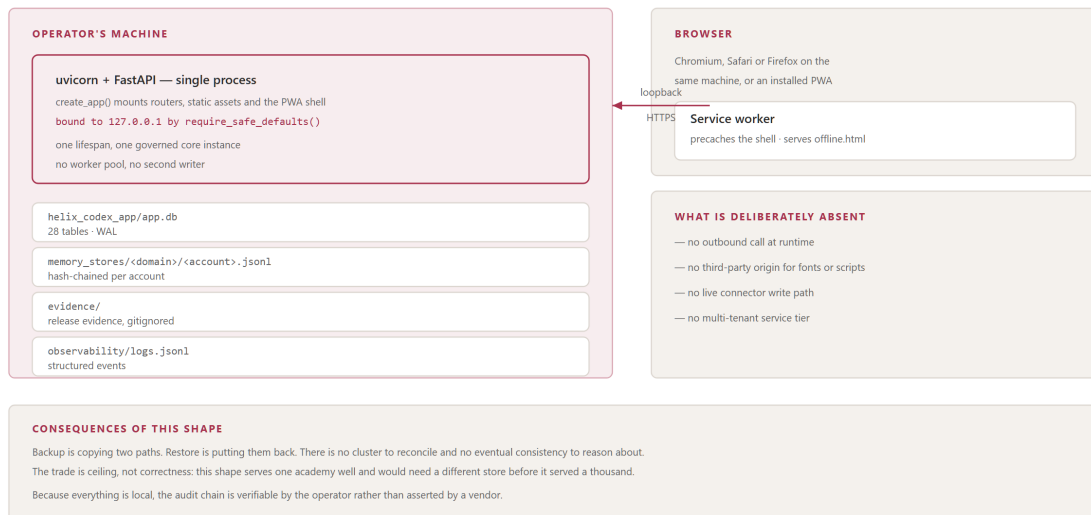


Figure 11: Runtime topology. The browser talks to a loopback socket; the process owns the store; nothing else is reachable.

Consequences

Backup is copying two paths and restore is putting them back, which is why the backup and restore gates are part of the release profile. The audit chain is verifiable by the operator directly. And the honest trade is ceiling rather than correctness: this shape serves one academy well and would need a different store before it served a thousand.

The installable shell is a real part of the runtime rather than a marketing surface. A service worker precaches the application shell, the offline page is served when a navigation fails, and the manifest declares the icons and display mode. The PWA assets are covered by their own release gate.

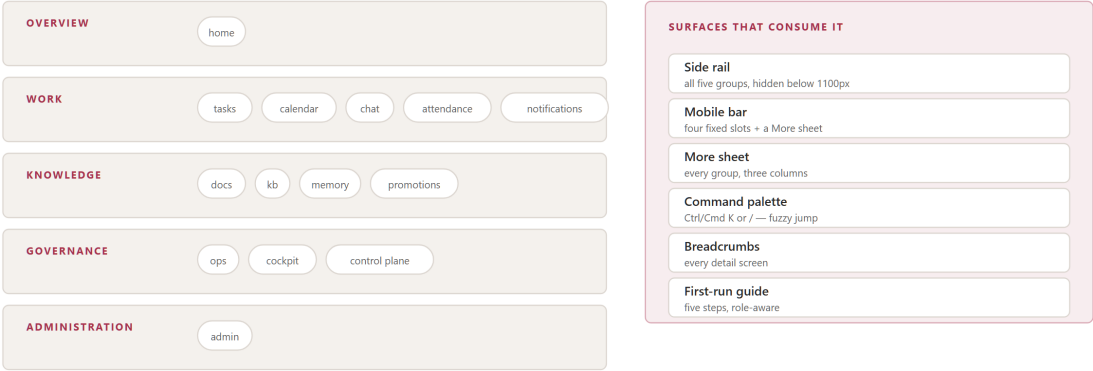
Navigation and information architecture

Four navigation surfaces — a desktop rail, a mobile bottom bar, a More sheet and a command palette — are all generated from one model. That is an architecture decision with a usability consequence: a screen cannot appear in one surface and be missing from

another, because there is only one list to add it to.

NAVIGATION AND INFORMATION ARCHITECTURE

One navigation model drives four surfaces, so a screen cannot appear in one place and not another



The model lives in one template — NAV_GROUPS — and the rail, the mobile bar, the More sheet and the palette all iterate it. Adding a screen means adding one entry. It then appears in all four surfaces with the right icon, role gate and one-line explanation. Role gating is data on each entry, so an employee never sees an Administration group rather than seeing it disabled. The palette also offers actions, not just places — verifying the audit chain is reachable without hunting for the screen that owns it. Chrome writes are withheld on the control plane, which promises to carry no form at all; that promise is asserted by a test.

Figure 12: The navigation model and the surfaces that consume it. Role gating is data on each entry, so an employee sees a shorter list rather than a list of disabled items.

Decisions taken for cognitive load

The interface is dark, single-themed and low-noise by intent. There is no light-mode toggle, because a second theme doubles the surface area to verify and the target environment is a dim office rather than daylight. Colour carries meaning rather than decoration: an amber pill means a decision is waiting, a sage pill means something was verified, and a clay pill means something is stopped.

Create forms are folded behind disclosure until needed, every screen states what it is for in one line, and empty states explain what would appear there and what to do to make it appear. A first-run checklist walks a new user through the five or six screens that matter, and remembers that it has been dismissed.

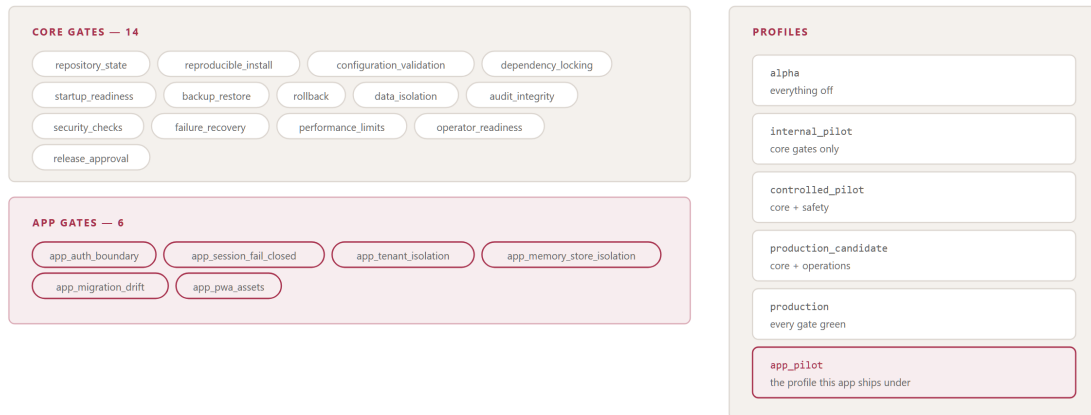
The command palette is the escape hatch for the cost of grouping: a user who knows the name of a screen never has to remember which group it lives in. It also exposes actions, not only destinations — verifying the audit chain is reachable without first finding the screen that owns it.

Release governance

What may ship is computed, not declared. Twenty gates report green or red; a profile states which subset matters; the classification falls out of that arithmetic. There is no path by which a build is talked into a higher label than its gates support.

RELEASE GOVERNANCE

Twenty gates, six profiles, and a classification that is computed rather than declared



Classification is derived: gates report green or red, the profile decides which subset matters, and the outcome falls out of that arithmetic.

A profile cannot be talked into a higher label. Requesting production while a production gate is red yields NOT_READY, not a warning.

The app ships under app_pilot, whose required set is the five safety-critical core gates plus all six app gates.

Gate results are written as evidence with a manifest naming the commit, so a release claim points at the run that produced it.

Running the gate with write_evidence=True regenerates release/release-manifest.json; tests pass write_evidence=False so a test run cannot move the release record.

This document was produced from a tree whose app tests passed 758 of 758; the two suite failures elsewhere are sandbox file-guard artifacts, not repo failures.

Figure 13: The gate set and the profiles. The app ships under app_pilot, whose required set is five safety-critical core gates plus all six app gates.

How to read the gate set

The six app gates are the ones specific to this product: the auth boundary is complete, sessions fail closed when revoked or expired, tenant isolation holds on reads, memory stores are not visible across accounts, the store matches its migration head, and the PWA assets are present and valid. Each of these is written so that it can also fail — the checks are exercised against a deliberately broken fixture, not merely asserted against a healthy one.

A gate that cannot fail is not a gate. Every check in this set has a companion test that breaks the condition and asserts the check reports red.

Gate results are written as evidence with a manifest naming the commit and the timestamp, so a release claim points at the run that produced it. Running the gate in evidence-writing mode regenerates the release manifest; the test suite explicitly suppresses that, so a test run cannot move the release record.

Non-functional characteristics

Property	Position	Evidence
Data residency	Everything stays on the operator's machine	No outbound call at runtime; fonts and scripts self-hosted
Data mode	Simulated throughout this build	data_mode is simulated_realistic on every record; no live connector write path
Auditability	Hash-chained per account, verifiable by the operator	verify_store_file() re-walks the chain; exposed as a user action
Failure mode	Fail closed	Governance controls raise rather than skip when their dependency is missing
Accessibility	Dark theme at WCAG AA for text	All foreground/background pairs computed; tightest text pair 4.90:1
Scale ceiling	One operator, one machine	Single process owns a SQLite file; not a multi-tenant tier
Backup / restore	Two paths, covered by gates	backup_restore and rollback are required profile gates
Browser support	Modern evergreen browsers	Service worker, native dialog, CSS custom properties

On the accessibility position

The interface is built dark-first and verified numerically rather than by eye. Every text and background pair in the palette was computed against WCAG AA, and the tinted pill backgrounds were composited over their real backdrop before measuring — measuring a translucent pill against black would have produced a flattering and wrong answer. The tightest passing pair sits at 4.90:1.

Risks, limits and recommendations

The following are the honest limits of the build as it stands. They are ordered by the cost of leaving them unaddressed.

Risks

1. **The scale ceiling is real and will be reached before the feature ceiling.** A single process owning one SQLite file is correct for one academy and wrong for a dozen. The migration is not difficult, but it is a migration, and it should be scheduled against a growth forecast rather than discovered during one.
2. **Simulated data has never met real data.** Every figure in this system is produced in simulated_realistic mode. The first real roster import is the moment the adapters meet messy inputs — absent guardians, mid-term enrolments, names that do not match the fee records. That is the highest-value next test.
3. **The permission matrix is coarse at the manager boundary.** A manager's admin scope is expressed as a single scoped value rather than as a set of org units. That is adequate for a flat academy and will not survive a second location with separate managers.
4. **Session lifetime and revocation are per-machine.** There is no device list and no remote sign-out, so a lost laptop is handled by revoking the account rather than the session. Acceptable at this scale; worth revisiting if the operator count grows.
5. **The interface has been reviewed at two viewports.** Desktop at 1440 and a true mobile width of 390 were checked by rendering the real pages. The 640px and 768px breakpoints were reasoned from the stylesheet rather than observed, and that gap should be closed before a wider rollout.

Recommendations, in order

1. Import one real roster and reconcile it against the simulated fixtures. Treat every discrepancy as a finding rather than a data-quality complaint.
2. Exercise backup and restore against a real store with real memory chains, and verify the chain after restore rather than before.
3. Review the intermediate breakpoints and the tablet layout, which is currently the least examined surface.
4. Decide the multi-location question before it is forced: either scope managers per org unit, or state explicitly that one deployment equals one location.
5. Record a written threshold — operator count, record volume, or concurrent users — at which the single-process deployment is revisited. A ceiling without a trigger is a ceiling nobody notices.

Appendix — route inventory

Every route mounted by the application, grouped by module. Routes under `/app` carry the account guard; routes marked with a dagger also carry the CSRF check. The health probe and the static mount are the only unguarded surfaces.

Surface	Routes
Shell	<code>/</code> and <code>/app</code> (home), <code>/app/healthz</code>
Identity	<code>/app/auth/login</code> , <code>/app/auth/logout †</code> , <code>/app/auth/password †</code> , <code>/app/auth/me</code>
Admin	<code>/app/admin</code> , <code>/users</code> , <code>/users/{id}</code> , <code>/domains †</code> , <code>/org-units †</code> , <code>/evidence/export</code>
Tasks	<code>/app/tasks</code> , <code>/tasks/{id}</code> , <code>/api/tasks/{id}</code> , <code>create †</code> , <code>comment †</code> , <code>update †</code>
Calendar	<code>/app/calendar</code> , <code>/api/events</code> , <code>/api/oncall</code> , <code>create †</code> , <code>rsvp †</code> , <code>cancel †</code>
Messaging	<code>/app/chat</code> , <code>/chat/{id}</code> , <code>/api/conversations</code> , <code>/messages</code> , <code>/stream (SSE)</code> , <code>send †</code>
Notifications	<code>/app/notifications</code> , <code>/api/notifications</code> , <code>/stream (SSE)</code> , <code>mark-read †</code>
Attendance	<code>/app/attendance</code> , <code>/api/attendance/records</code> , <code>/summary</code> , <code>punch †</code>
Docs / KB	<code>/app/docs</code> , <code>/docs/{id}</code> , <code>/app/kb</code> , <code>/api/documents/{id}</code> , <code>/versions</code> , <code>block edits †</code>
Memory	<code>/app/memory</code> , <code>/memory/promotions</code> , <code>/memory/proposals/{id}</code> , <code>/memory/ledger/verify</code> , <code>propose †</code> , <code>review †</code> , <code>rollback †</code>
Ops	<code>/app/ops</code> , <code>/ops/{engine_id}</code> , <code>/api/ops/workflows/{id}</code> , <code>/stream</code> , <code>submit †</code>
Cockpit	<code>/app/cockpit</code> , <code>/owner</code> , <code>/coach</code> , <code>/parent</code> , <code>/control-plane</code> , <code>/api/cockpit/summary</code>
Low-code	<code>/app/api/packs</code> , <code>/api/sections</code> , <code>register †</code> , <code>activate †</code>

The route table is also a governance surface. The control plane is the clearest case: it is a read-only panel whose template is required by test to carry no form element at all, so that no write affordance can appear on a screen that promises to only display. That requirement is asserted rather than documented, which is the difference between a promise and a guarantee.